



信息与软件工程学院

课程项目报告

目

录

课程名称： 基于人工智能的图像处理技术（PBLF）

学 期： 2025-2026-2

授课教师： 甘 涛

序号	学号	姓名
1（组长）	2025090911029	蓝雅然
2	2025090911026	田小溪
3	2025090912004	孙诚

第 1 章 项目概述.....	3
-----------------	---

1.1 项目任务	3
1.2 项目分工	3
第 2 章 复杂软件工程问题的解决	4
2.1 系统分析和设计	4
2.1.1 车牌识别	4
2.2.1 图像分类	4
2.2 编码和测试	4
2.1.1 车牌识别	4
2.2.1 图像分类	5
第 3 章 项目总结	6
3.1 项目完成情况	6
3.2 收获和体会	6

****注**格式要求，按模板格式：**正文中文统一用宋体小四，英文用 times new roman 小四。正文间距 20 磅，图标题和表标题为五号字体，图标题位于图片下方，表标题位于表格上方。

重点针对第二章和第三章 PPT 中的“项目任务”进行分析、设计、编码和测试，将其过程和结果，归纳写入报告相应章节中。可不局限于提出的问题，鼓励发挥和扩展内容。

第1章 项目概述

1.1 项目任务

本项目围绕"基于人工智能的图像处理技术"展开，包含两个子任务：车牌识别和图像分类。这两个任务分别代表传统计算机视觉方法和深度学习方法。

(1) 车牌识别

给定包含车辆车牌的图像，我们的目标是自动识别并输出车牌号码。整体思路是按照"预处理 → 车牌定位 → 字符分割 → 字符识别"四个阶段推进，主要用 OpenCV 实现图像处理，用 SVM 模型辅助判断候选区域是否真的是车牌，同时也尝试了 YOLOv8 做了初步对比。

(2) 图像分类

我们用 PyTorch 框架实现了三个分类实验——鸢尾花三分类、猫狗二分类和 MNIST 手写数字识别，逐步从简单的全连接网络过渡到卷积神经网络，最后尝试了迁移学习。在这其中进行了多种尝试：分批次、GPU 加速、数据打乱、数据增强、从零训练 VGG 以及用现成 VGG”。

1.2 项目分工

本项目有 3 名同学协助完成，其分工明细如表 1 所示。

表 1 项目分工明细

序号	姓名	项目分工
1	蓝雅然	图像分类中猫狗分类与文档修改实施迁移学习方案，整理各实验数据，完成最终文档
2	田小溪	Doxc，图像分类中鸢尾花与 MNIST，实现 BP 神经网络和 CNN；编写分类部分报告
3	孙诚	PPT 制作，车牌识别，阅读代码并且修改部分代码，收集相应实验数据；优化图像识别的部分代码，编写实验报告对应部分。

第 2 章 复杂软件工程问题的解决

2.1 系统分析和设计

2.1.1 车牌识别

【任务分析】

本项目最初的直觉是：车牌颜色明显、形状规则，应该可以比较容易地从车辆图像中定位出来。但在实际处理过程中发现，真实图像中的干扰因素较多，例如车身反光、路面纹理、背景边缘、车身装饰线条等，都会在边缘检测和轮廓提取时形成类似车牌的矩形区域，导致程序容易选出“假车牌”。此外，普通蓝牌和新能源绿牌在颜色、字符数量和版式上存在差异，新能源车牌还可能出现分隔点干扰字符分割。因此，本项目的核心工程问题不是单纯“找到一个矩形”，而是要在复杂背景中逐步缩小候选范围，并根据车牌类型选择不同的处理规则。

【总体方案】

系统整体流程分为四个阶段：图像输入与预处理、车牌区域定位、字符分割、字符识别。首先读取原始车辆图像，对图像进行灰度化、去噪、边缘提取、二值化和形态学处理；然后通过颜色特征和几何特征定位车牌区域；接着对车牌图像进行二值化和字符分割，将完整车牌拆分为单个字符；最后使用模板匹配方法识别每个字符，并结合车牌字符位置规则进行修正，最终输出完整车牌号。

【车牌定位方案设计】

方案一是基于颜色特征的车牌定位。程序先将图像转换到 HSV 颜色空间，分别提取蓝色和绿色区域，再通过闭运算、开运算和中值滤波连接车牌主体、去除小噪声，最后根据候选区域的宽高比和面积筛选最可能的车牌区域。该方案对蓝牌和新能源绿牌都比较直接，尤其适合车牌颜色明显的图像，因此在本项目中被作为优先方案。

方案二是基于边缘和几何约束的备用定位方案。当颜色定位失败时，程序会先将图像灰度化并进行高斯去噪，然后使用 Sobel 算子提取水平方向边缘，再经过 Otsu 二值化、形态学闭运算、开运算和中值滤波，将车牌字符区域尽量连接成一个整体。之后通过轮廓检测获取候选矩形，并根据宽高比、面积大小等几何条

件筛选出最像车牌的区域。该方案不依赖车牌颜色，能够作为颜色定位失败时的兜底方案。

最终采用的定位策略是“两级定位”：优先使用 HSV 颜色定位快速锁定蓝牌或绿牌；如果颜色定位没有找到合适区域，再使用 Sobel 边缘检测和形态学处理进行备用定位。这样既利用了车牌颜色特征，又保留了传统边缘检测方法的兼容性，提高了系统在不同图像条件下的稳定性。

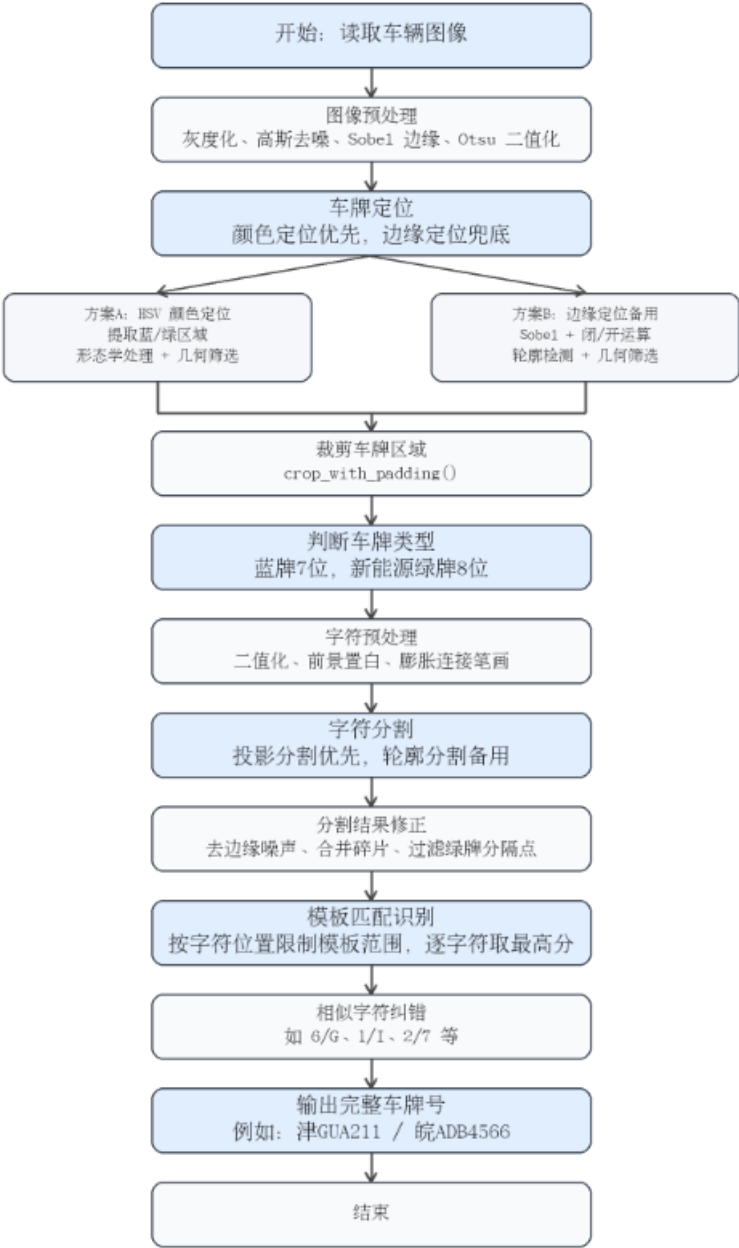


图 1 车牌识别系统流程

【测试样例】实验使用两张真实场景照片（蓝牌 GUA.jpg、绿牌 ADB.jpg）进行验证。



图 1 GUA. jpg (蓝牌)



图 2 ADB. jpg (绿牌)



图 3、图 4 分别为两者 debug 时的场景

2.2.1 图像分类

图像分类部分包含三个实验，复杂度逐渐递增：从结构化的低维数据（鸢尾花），到小规模自然图像（猫狗），再到标准基准数据集（MNIST）。我们的思路是先把最基础的东西跑通，再逐步引入更复杂的模型和训练技巧。

【鸢尾花分类】我们使用了 $4 \rightarrow 20 \rightarrow 3$ 的三层全连接网络：输入层对应 4 个特征，中间一层 20 个神经元加 ReLU 激活，输出层 3 个节点对应 3 个类别。损失函数用 CrossEntropyLoss，优化器用 Adam (lr=0.01)。这个实验我们迭代了四个版本（iris0~iris3），每版加一个新机制，目的是搞清楚每个改动对结果的影响。实验逐步引入 DataLoader、训练/测试集划分和 GPU 加速。

鸢尾花分类神经网络流程图

环境: Python 3.7.9 PyTorch 1.7.0



【猫狗分类】依次尝试浅层 CNN、自定义 VGG16 和预训练 VGG16 迁移学习三种方案，比较不同模型与训练策略对二分类效果的影响。



【MNIST 手写数字识别】网络构建了两层卷积（Conv→ReLU→MaxPool）加两层全连接分类器的 CNN，中间加了 Dropout(0.5) 防止过拟合。目标是尽量接近 MNIST 经典基准水平。



2.2 编码和测试

2.1.1 车牌识别

本实验的代码是在原有车牌识别程序基础上修改完成的。原程序的基本流程是：先对图像进行灰度化、滤波、Sobel 边缘提取、阈值化和形态学处理，再通过轮廓定位车牌区域，最后进行字符分割和模板匹配识别。我们保留了这条主流程，但没有继续使用 SVM 作为候选区域判定方法，而是改成了更加直接的规则筛选和颜色辅助定位。

（1）车牌定位模块

在车牌定位部分，程序首先对图像进行灰度化和高斯去噪，然后使用 Sobel 算子提取水平方向的边缘特征。由于车牌中字符较多，边缘信息比较密集，经过二值化和形态学闭运算后，车牌区域会更容易连成一个整体。随后程序通过轮廓检测得到候选区域，并根据宽高比、面积大小等规则筛选出最像车牌的矩形区域。

相比原版本，我们去掉了 SVM 判定步骤，没有再把候选区域送入预训练模型分类，而是使用“轮廓几何特征 + 面积评分”的方式选择车牌。同时，为了适应新能源绿牌，程序增加了基于 HSV 颜色空间的定位方法：先提取蓝色和绿色区域，如果能够通过颜色特征找到车牌，就优先使用颜色定位结果；如果失败，再回到边缘和轮廓定位流程。

（2）字符分割模块

原版本主要依靠轮廓查找来分割字符，但这种方法在新能源绿牌上容易出问题。因为绿牌通常是 8 位字符，并且第 2 位后面有分隔点，如果仍然按普通蓝牌的 7 位字符处理，就会出现多分割、少分割或字符错位。

因此，我们在修改后加入了车牌类型判断：如果检测到车牌绿色区域比例较高，就按新能源车牌处理，期望字符数为 8；否则按普通蓝牌处理，期望字符数为 7。字符分割时优先使用垂直投影法，根据每一列前景像素数量的变化寻找字符

区间，并对边缘噪声、过窄区域和新能源车牌分隔点进行过滤。如果投影分割失败，再退回到轮廓分割方法。

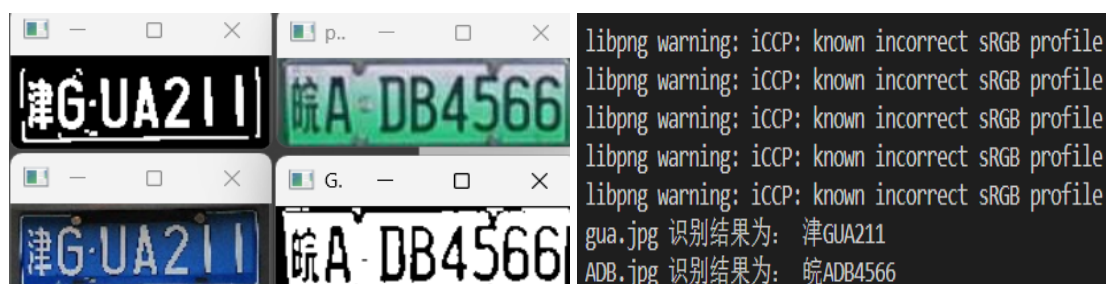
（3）字符识别模块

字符识别部分仍然采用模板匹配。程序将分割出的字符与模板库中的数字、字母和省份简称逐个比较，取匹配得分最高的字符作为识别结果。

相比原版本，修改后的代码增加了字符位置约束。比如第 1 位只能匹配省份简称，第 2 位主要匹配字母；对于普通蓝牌，后几位主要限制在数字范围；对于新能源绿牌，最后两位也优先按数字处理。这样可以减少明显不合理的识别结果，例如把数字误识别成形状相近的字母。

（4）测试结果与分析

实验中使用两张真实车牌图片进行测试。普通蓝牌 `gua.jpg` 最终识别结果为“津 GUA211”，新能源绿牌 `ADB.jpg` 最终识别结果为“皖 ADB4566”。



最开始按照原版本流程处理时，普通蓝牌可以较好识别，但新能源绿牌容易失败，主要原因是绿牌字符数为 8 位，并且存在分隔点，原来的字符分割和模板匹配规则并不适配。经过修改后，程序增加了蓝绿牌颜色判断、7 位/8 位字符数量判断、投影分割修正和模板匹配范围限制，因此最终能够同时识别普通蓝牌和新能源绿牌。

（5）识别成功的关键改进模块

本实验并不是直接使用原程序就完成了识别，而是在原有流程上进行了多处针对性修改。原程序主要面向普通蓝牌，基本流程是边缘检测、轮廓定位、字符分割和模板匹配，并且预留了 SVM 候选区域判断功能。但在我们的实现中，没有

继续依赖 SVM，而是改用规则筛选、颜色辅助定位和模板范围约束来完成识别。

为了让程序能够同时识别普通蓝牌和新能源绿牌，我们主要做了以下几方面改进。

首先，增加了蓝牌和绿牌的颜色判断。程序在 HSV 颜色空间中分别提取蓝色和绿色区域，如果能通过颜色特征定位到车牌，就优先使用颜色定位结果。这样可以避免只依赖边缘轮廓时，把背景中的其他长条区域误判为车牌。

其次，增加了车牌类型判断。普通蓝牌一般是 7 位字符，而新能源绿牌通常是 8 位字符。修改后，程序会根据绿色像素比例判断是否为新能源车牌。如果是绿牌，就按照 8 位字符处理；如果是蓝牌，就按照 7 位字符处理。这个修改解决了原程序在绿牌上容易少分割或错分割的问题。

第三，改进了字符分割方法。原程序主要依靠轮廓分割字符，但新能源绿牌中间存在分隔点，容易被误当成一个字符。修改后，程序优先使用垂直投影法，根据每一列前景像素数量寻找字符区域，并增加了边缘噪声过滤、窄区域合并、分隔点去除等规则。如果投影分割失败，再退回到轮廓分割方法，提高了分割的稳定性。

第四，限制了模板匹配范围。原程序在识别每个字符时，会在数字、字母和省份简称中统一匹配，容易把形状相近的字符混淆。修改后，程序根据车牌位置限制候选模板范围：第 1 位只匹配省份简称，第 2 位主要匹配字母，后几位根据蓝牌或绿牌规则限制为数字或字母数字组合。这样可以减少明显不符合车牌格式的错误结果。

最后，针对容易混淆的字符进行了简单纠错。例如数字 6 和字母 G、数字 1 和其他窄字符、数字 2 和相似字符之间容易混淆，程序通过字符位置、宽高比例和像素分布特征进行辅助判断，从而提高最终识别结果的准确性。

因此，最终识别成功并不是依靠单一算法完成的，而是通过“颜色定位 + 车牌类型判断 + 7/8 位字符规则 + 投影分割修正 + 模板范围限制 + 易混字符纠错”共同实现的。经过这些修改后，程序最终能够正确识别普通蓝牌 `gua.jpg`，输出“津 GUA211”；也能够正确识别新能源绿牌 `ADB.jpg`，输出“皖 ADB4566”。

总体来看，本实验是在传统图像处理方法的基础上完成车牌识别流程的改进。修改后的程序不依赖 SVM，而是通过颜色特征、轮廓规则、投影分割和位置约束模板匹配提高了识别效果。不过，该方法仍然依赖人工规则，对复杂光照、倾斜车牌、污损车牌和复杂背景的适应能力有限，后续可以考虑使用 YOLOv8 等目标检测模型进一步提高鲁棒性。

2.2.1 图像分类

(1) 鸢尾花分类实验

我们把鸢尾花实验分成了四个版本迭代，每次只改一个地方，这样能清楚地看到每个改动的作用。

iris0（最简版本）：直接把全部 150 条数据都拿来训练，没有测试集。跑 10 个 Epoch 之后损失就降下去了，准确率能到 96% 以上。但这个版本的准确率其实没什么参考价值，因为训练集和“测试集”是同一批数据，模型见过所有样本。

iris1（引入 DataLoader）：加了 DataLoader 做批处理（batch_size=16），把数据加载流程规范化。代码结构更清晰，也为后续版本打了基础。

iris2（划分训练/测试集）：按 4:1 把数据拆成 120 条训练集和 30 条测试集，并且在训练时显式切换 net.train() 和 net.eval() 模式。这是这个系列实验里最重要的一步——有了真正意义上的测试集，评估结果才有参考价值。测试集准确率稳定在 93.33%。

iris3（GPU 加速）：在 iris2 的基础上把模型和数据都 .to(device) 迁移到 GPU。准确率和 iris2 完全一样，但训练速度快了不少。说明对于这种小数据集，GPU 的优势主要体现在速度上，不影响收敛结果。

核心训练循环如下（以 iris3 为例）：

```
optimizer = torch.optim.Adam(net.parameters(), lr=0.01)
for epoch in range(80):
    net.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        pred = net(x)
        loss = criterion(pred, y)
        optimizer.zero_grad()
        loss.backward()
```

```
optimizer.step()

net.eval()

with torch.no_grad():
    # 在测试集上计算准确率

    correct = (net(X_test).argmax(1) == y_test).sum()

    acc = correct / len(y_test)
```

值得一提的是，我们最初用的是 SGD 优化器，收敛很慢，后来换成 Adam 之后明显快多了，说明对于小网络 Adam 确实比 SGD 更友好。

表 3 鸢尾花分类各版本实验结果

版本	结果	主要改动	说明
iris0	Epoch:77/80, Loss:0.2002, Accuracy:98.0000% Epoch:78/80, Loss:0.1968, Accuracy:98.0000% Epoch:79/80, Loss:0.1934, Accuracy:98.0000% Epoch:80/80, Loss:0.1901, Accuracy:98.0000% 鸢尾花预测准确率为:0.9800%	基础全量训练	无测试集, 准确率无泛化意义
iris2	Epoch 76/80: Train Loss is:0.1088, Train Accuracy is:98.3333%, Test Accuracy is:96.6667 Epoch 77/80: Train Loss is:0.1069, Train Accuracy is:98.3333%, Test Accuracy is:96.6667 Epoch 78/80: Train Loss is:0.1051, Train Accuracy is:98.3333%, Test Accuracy is:96.6667 Epoch 79/80: Train Loss is:0.1033, Train Accuracy is:98.3333%, Test Accuracy is:96.6667 Epoch 80/80: Train Loss is:0.1016, Train Accuracy is:98.3333%, Test Accuracy is:96.6667	划分训练/测试集	首次具备真实评估能力, 结果可信
iris3	Epoch 77/80: Train Loss is:0.1235, Train Accuracy is:97.5000%, Test Accuracy is:100.0000 Epoch 78/80: Train Loss is:0.1214, Train Accuracy is:97.5000%, Test Accuracy is:100.0000 Epoch 79/80: Train Loss is:0.1194, Train Accuracy is:97.5000%, Test Accuracy is:100.0000 Epoch 80/80: Train Loss is:0.1175, Train Accuracy is:97.5000%, Test Accuracy is:100.0000	加入 GPU 加速	精度不变, 训练速度提升

(2) 猫狗二分类实验

猫狗分类是这次项目里花时间最多的部分。我们先后试了三套方案。

方案一：浅层 CNN（效果差，但让我们理解了问题所在）

第一版我们用了一个 3 层卷积的浅层网络，图像缩放到 28×28 。跑出来验证集准确率只有大约 50%，和随机猜基本没区别。仔细分析发现有两个问题：一是 28×28 太小了，猫和狗的纹理细节基本都丢失了；二是网络太浅，就算输入图像质量够，这个网络也没能力提取足够有区分性的特征。

方案二：从头训练 VGG16（有改善，但还不够）

第二版我们搭了一个 VGG16 结构，输入图像换成 224×224，从头开始随机初始化训练。准确率提升到了大约 80%，但收敛很慢，跑了很多个 Epoch 也没见明显提升。主要原因有两个：一是我们的数据集规模不大，随机初始化的权重需要大量数据才能收敛到合理状态；二是当时我们把权重冻结的位置搞错了，把一部分分类头的参数也冻住了，导致模型可学习的参数太少。

方案三：预训练 VGG16 + 迁移学习（最终方案）

第三版加载了 ImageNet 预训练权重，冻结所有卷积层，只替换并训练最后的分类头（把原来的 1000 类输出改成 2 类）。代码核心如下：

```
vgg16 = models.vgg16(pretrained=True)

# 冻结卷积层，不更新这部分参数
for param in vgg16.features.parameters():
    param.requires_grad = False

# 把分类头的最后一层换成二分类
vgg16.classifier[-1] = nn.Linear(4096, 2)

# 只让分类头参数参与梯度更新
optimizer = torch.optim.Adam(
    vgg16.classifier.parameters(), lr=1e-4)
```

结果让我们有点意外——只训练了 5 个 Epoch，验证集准确率就到了 100%。这说明 ImageNet 预训练权重学到的低级视觉特征（比如边缘、纹理、颜色斑块）对猫狗识别任务完全适用，不需要重新学，只需要在顶部加一个简单的分类器就够了。

表 4 猫狗二分类三种方案对比

方案	输入尺寸	验证集最高准确率	问题与分析
方案一： 浅层 CNN	28×28	Epoch1/5: Train Accuracy:45.5000%,Valid Accuracy:50.0000% Epoch2/5: Train Accuracy:56.0000%,Valid Accuracy:50.0000% Epoch3/5: Train Accuracy:58.5000%,Valid Accuracy:60.0000% Epoch4/5: Train Accuracy:59.0000%,Valid Accuracy:60.0000% Epoch5/5: Train Accuracy:61.0000%,Valid Accuracy:50.0000%	图像缩放太小，网络过浅，特征提取能力严重不足
方案二： 自定义 VGG16 （随机初始化）	224×224	Epoch26/30: Train Accuracy:94.5000%,Valid Accuracy:70.0000% Epoch27/30: Train Accuracy:91.5000%,Valid Accuracy:80.0000% Epoch28/30: Train Accuracy:93.5000%,Valid Accuracy:60.0000% Epoch29/30: Train Accuracy:75.5000%,Valid Accuracy:70.0000% Epoch30/30: Train Accuracy:94.5000%,Valid Accuracy:90.0000% 最佳训练次数30/30,最佳验证损失0.4852	数据不足导致收敛慢；权重冻结位置有误

方案	输入尺寸	验证集最高准确率	问题与分析
方案三： 预训练 VGG16 (迁移 学习)	224×224	Epoch6/10: Train Accuracy:100.0000%,Valid Accuracy:100.0000% Epoch7/10: Train Accuracy:100.0000%,Valid Accuracy:100.0000% Epoch8/10: Train Accuracy:100.0000%,Valid Accuracy:100.0000% Epoch9/10: Train Accuracy:100.0000%,Valid Accuracy:100.0000% Epoch10/10: Train Accuracy:100.0000%,Valid Accuracy:100.0000% 最佳训练次数9/10,最佳验证损失0.0107 训练时间: 14.007251977920532	预训练特征 直接可用， 收敛极快

可见，迁移学习在小数据集上的优势明显，直接利用 ImageNet 预训练权重可以大幅提升模型的泛化能力。

(3) MNIST 手写数字识别实验

MNIST 是相对成熟的任务，我们主要用它来练习 CNN 的标准写法，同时验证 Dropout 对过拟合的抑制效果。

网络结构如下：

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), # 1→32 通道
            nn.ReLU(),
            nn.MaxPool2d(2)                # 28×28 → 14×14
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(32, 64, 3, padding=1), # 32→64 通道
            nn.ReLU(),
            nn.MaxPool2d(2)                # 14×14 → 7×7
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*7*7, 128),
            nn.ReLU(),
            nn.Dropout(0.5), # 随机关闭 50%神经元，防过拟合
```

```
nn.Linear(128, 10)
)
```

训练配置: Adam 优化器 (lr=0.001)、CrossEntropyLoss、batch_size=64、共训练 5 个 Epoch。

表 5 MNIST 训练过程结果

```
Epoch 1/5: Train Loss is:0.1481, Train Accuracy is:95.6800%, Test Accuracy is:98.1300
Epoch 2/5: Train Loss is:0.0469, Train Accuracy is:98.5617%, Test Accuracy is:98.3200
Epoch 3/5: Train Loss is:0.0316, Train Accuracy is:99.0317%, Test Accuracy is:98.6700
Epoch 4/5: Train Loss is:0.0242, Train Accuracy is:99.2333%, Test Accuracy is:98.6700
Epoch 5/5: Train Loss is:0.0191, Train Accuracy is:99.3733%, Test Accuracy is:98.8000
```

有一个有趣的现象: 第 1 个 Epoch 测试集准确率 (98.21%) 就已经比训练准确率 (96.00%) 高了。这是因为 Dropout 在训练时会随机关掉 50% 的神经元, 相当于给训练带来了"人为干扰", 而推断时不启用 Dropout, 所以测试阶段的模型比训练阶段"更完整", 准确率反而更高。这也验证了 net.train() / net.eval() 切换的必要性。

第 3 章 项目总结

3.1 项目完成情况

3.1.1 总体情况

车牌识别部分，完成了从图像预处理、车牌定位、字符分割到模板匹配识别的完整流程，并在原有代码基础上进行了修改，使程序能够同时识别普通蓝牌和新能源绿牌。

图像分类部分侧重于深度学习方法，完成了鸢尾花三分类、猫狗二分类和 MNIST 手写数字识别三个实验，分别验证了全连接神经网络、卷积神经网络和迁移学习方法的效果。

总体来看，本项目的五个子任务均达到了基本预期，完成了从需求分析、算法设计、代码实现到实验测试的全过程。

其中，车牌识别部分成功识别了普通蓝牌 `gua.jpg`，输出结果为“津 GUA211”；同时通过增加蓝绿牌颜色判断、7 位/8 位字符数量判断、投影分割修正和模板匹配范围限制，也成功识别了新能源绿牌 `ADB.jpg`，输出结果为“皖 ADB4566”。图像分类部分中，鸢尾花三分类测试集准确率达到 93.33%，猫狗二分类在迁移学习方案下验证集准确率达到 100%，MNIST 手写数字识别测试集准确率达到 98.82%。

（1）蓝雅然

蓝雅然主要负责图像分类中的猫狗二分类实验、迁移学习方案实施、实验数据整理以及最终文档的修改完善。猫狗分类部分先后尝试了浅层 CNN、自定义 VGG16 和预训练 VGG16 迁移学习三种方案，并对不同模型结构和训练策略的效果进行了对比分析。

在实验过程中，浅层 CNN 方案由于输入图像尺寸较小、网络结构较简单，验证集准确率接近随机猜测；自定义 VGG16 方案相比浅层 CNN 有所提升，但从零训练对数据量要求较高，收敛速度较慢，准确率提升有限。最终采用预训练 VGG16 迁移学习方案，冻结卷积特征提取层，只训练最后的分类头，在 5 个 Epoch 后验证集准确率达到 100%。

通过这一部分工作，完成了猫狗二分类任务的模型搭建、训练测试和结果分析，也验证了迁移学习在小规模图像分类任务中的有效性。同时，蓝雅然还参与整理各实验数据和最终文档内容，使不同实验模块的结果能够统一呈现在报告中。

（2）田小溪

田小溪主要负责图像分类中的鸢尾花三分类实验和 MNIST 手写数字识别实验，并参与分类部分报告内容的编写。鸢尾花分类部分使用全连接神经网络完成三分类任务，网络输入为 4 个鸢尾花特征，输出为 3 个类别。实验过程中逐步迭代了多个版本，从最基础的全量训练，到引入 DataLoader，再到划分训练集和测试集，最后加入 GPU 加速，逐步完善了训练和评估流程。

最终，鸢尾花分类实验在测试集上取得了 93.33% 的准确率。MNIST 手写数字识别部分使用卷积神经网络完成 0—9 共 10 类数字分类，网络结构包括卷积层、ReLU 激活、最大池化、全连接层和 Dropout。经过 5 个 Epoch 训练后，MNIST 测试集准确率达到 98.82%。

通过这两部分工作，完成了从低维结构化数据分类到图像分类任务的实践，验证了全连接神经网络和卷积神经网络在不同类型数据上的应用效果，也为图像分类部分提供了比较完整的实验支撑。

（3）孙诚

主要负责车牌识别部分、部分代码阅读与修改、实验数据整理，以及报告中车牌识别相关内容的编写。

车牌识别部分是在原有代码基础上进行修改和完善的。原程序整体流程比较清楚，主要包括图像预处理、车牌定位、字符分割和模板匹配识别，但它更适合普通蓝牌，对新能源绿牌的适配不足，并且原代码中虽然预留了 SVM 候选区域判断功能，但本项目最终没有继续使用 SVM，而是改用颜色判断、轮廓规则、投影分割和模板匹配范围约束来完成识别。

在具体修改过程中，针对新能源绿牌识别失败的问题，程序增加了 HSV 颜色空间下的蓝绿牌判断；根据普通蓝牌 7 位、新能源绿牌 8 位的差异，加入了字符数量判断；针对新能源绿牌第 2 位后的分隔点问题，改进了投影分割和噪声过滤规则；针对数字与字母容易混淆的问题，增加了基于字符位置的模板匹配范围限制。

经过修改后，程序最终能够正确识别普通蓝牌和新能源绿牌。其中，蓝牌 gua.jpg 的识别结果为“津 GUA211”，绿牌 ADB.jpg 的识别结果为“皖 ADB4566”，完成了车牌识别任务的主要目标。

3.2 收获和体会

3.2.1 蓝雅然——从“堆模型”到“选对方案”

（1）模型不是越复杂越好

在猫狗分类这一部分，最大的体会是：图像分类任务不能只看模型名字大不大，还要看数据规模、输入尺寸和训练方式是否匹配。最开始使用浅层 CNN 时，模型效果并不好，验证集准确率接近随机猜测。这个结果一开始看起来有点挫败，但分析后发现， 28×28 的输入尺寸对于猫狗这种自然图像来说太小了，很多纹理和轮廓信息都被压缩掉了；同时网络结构也比较浅，特征提取能力不足，所以很难真正区分猫和狗。

（2）从零训练大模型并不一定适合小数据集

后来尝试自定义 VGG16 后，准确率有一定提升，但训练过程并不理想。一方面是因为 VGG16 参数量较大，如果从零开始训练，需要更多数据和更长时间；另一方面，实验中也出现过权重冻结位置不合适的问题，导致模型可训练参数不足，准确率长时间上不去。

这个过程让我意识到，深度学习实验不是把模型结构写出来就结束了。参数是否更新、优化器怎么设置、数据量是否足够、输入图像尺寸是否合理，都会直接影响最终结果。很多时候，模型本身没有错，但训练策略不合适，结果也会很差。

（3）迁移学习的优势很明显

最终采用预训练 VGG16 迁移学习后，效果提升非常明显，5 个 Epoch 后验证集准确率达到 100%。这个结果让我比较直观地理解了迁移学习的价值：预训练模型已经在大规模数据集上学到了通用的边缘、纹理和形状特征，在小数据集任务中不一定需要从零开始训练，只需要根据当前任务调整最后的分类层，就能得到较好的效果。

这也让我认识到，在实际工程中，选择合适方案比盲目使用复杂模型更重要。对于小规模数据集，从零训练一个大模型并不一定是最优选择，合理使用预训练模型反而更稳定、更高效。

（4）报告整理也需要逻辑

在整理文档时，我也发现，实验报告不能只写最终准确率，还要说明为什么这个方案有效、其他方案为什么效果不好，以及中间遇到的问题是怎样解决的。这样报告才不是简单罗列结果，而是能够体现完整的实验过程。通过这次项目，我对猫狗分类任务、迁移学习方法以及实验结果分析都有了更具体的认识。

3.2.2 田小溪——规范训练比单纯跑出准确率更重要

（1）高准确率不一定代表模型真的好

在鸢尾花分类和 MNIST 实验中，我最大的收获是对神经网络训练流程有了更完整的认识。鸢尾花数据集虽然规模很小，但非常适合用来理解一个机器学习实验应该怎么规范地完成。最开始直接把全部数据拿来训练时，准确率看起来很高，但这个结果其实没有太大参考价值，因为模型已经见过所有样本，没有真正独立的测试集。

后来划分训练集和测试集之后，才真正能够评价模型在未见数据上的表现。这让我意识到，训练准确率高并不代表模型真的学得好，只有在测试集上仍然表现稳定，结果才更可信。

（2）训练流程中的细节很重要

在实验过程中，`DataLoader`、`batch_size`、`train/eval` 模式切换、GPU 加速这些内容看起来只是代码细节，但它们实际上决定了实验流程是否规范。比如如果没有正确切换训练模式和测试模式，`Dropout` 等层的行为就会不一样，实验结果也可能不稳定；如果模型和数据没有放在同一个设备上，程序还会直接报错。

这些问题让我意识到，深度学习不是只写一个网络结构就可以了，还需要把数据准备、训练过程、测试过程和结果分析都处理好。只有流程规范，最后得到的准确率才有意义。

（3）CNN 更适合处理图像任务

MNIST 实验让我更加直观地理解了卷积神经网络为什么适合处理图像。手写数字虽然看起来简单，但如果直接把图像当成普通向量处理，就容易丢失局部空间结构；而 CNN 可以通过卷积核提取局部特征，再通过池化降低维度，最后用全连接层完成分类。最终测试集准确率达到 98.82%，说明 CNN 对这类图像识别任务确实非常有效。

（4）从会写代码到理解实验

这次实验中也遇到了一些细节问题，比如模型训练和测试阶段的状态切换、`Dropout` 在训练和推理时表现不同、数据和模型是否放在同一个设备上等。刚开始这些问题容易被忽略，但一旦写错，实验结果就会不稳定，甚至直接报错。

通过这次项目，我感觉自己不只是会调用 `PyTorch` 写网络了，更重要的是理解了一个深度学习实验从数据准备、模型搭建、训练、测试到结果分析的完整流程。总体来说，这部分实验让我认识到，深度学习不是简单堆网络结构，而是一个需要严谨实验流程支撑的工程任务。模型结构、数据划分、训练方式和评价指标都需要对应起来，最后得到的结果才有意义。

3.2.3 孙诚——为 AI 辅助编程，提供 context 和 Harness

(1) 先学会 debug，再谈写代码

在车牌识别这一部分，我最大的体会是：要做好一个程序，首先要学会 debug。程序不是单靠“看懂代码”就能跑通的，真正重要的是能够观察运行结果，根据现象判断问题出在哪一步。

最开始程序能够较好识别蓝牌，但在绿牌上会出现定位不准、字符数量不对、分隔点被误分割、数字和字母混淆等问题。如果只看最后输出的车牌号，很难知道到底是哪一步错了。因此，调试的关键不是盲目改代码，而是先把程序运行过程拆开，看清楚每一步的输入和输出。

(2) 用中间结果建立调试现场

为了定位这些问题，我通过 `cv2.imshow()` 不断查看中间处理结果，例如原图、定位出的车牌区域、二值化后的字符图像、字符分割结果等。通过这些中间图像，我逐步判断出问题分别出在车牌定位、字符分割和模板匹配几个环节，而不是简单地把最终错误归结为“程序识别失败”。

这个过程让我意识到，图像处理程序的调试非常依赖中间结果。因为每一步处理都会影响下一步，如果前面的车牌区域没有定位准确，后面的字符分割和模板匹配就很难正确；如果字符分割错了，即使模板匹配写得再好，最后输出也会出问题。

(3) AI 的作用是放大人的工程能力

这次项目也让我对 AI 辅助编程有了更实际的认识。现在的 AI 已经不只是简单回答问题，而是可以像一个编程助手一样，帮助理解代码、分析错误原因、提出修改方案，甚至协助完成局部代码调整。

但是要让 AI 真正发挥作用，前提是人要给它足够清楚的上下文。比如我不能只说“绿牌识别错了”，而是要告诉它：当前车牌区域是否定位正确、字符是否分割成了 8 位、哪一位被误识别、实际中间图像和预期结果有什么差别。这样 AI 才能基于具体现场给出有效建议。

(4) 人的价值在于提供上下文和验证闭环

从这个角度看，我在项目中的价值并不是单纯手动改几行代码，而是把一个模糊的问题转化成可以被分析、可以被修改、可以被验证的工程问题。具体来说，就是先用 `cv2.imshow()` 建立调试现场，再把中间结果和错误现象整理成清楚的上下文，然后让 AI 在这个基础上给出尽量小的代码修改，最后再由我运行程序验证修改是否有效。

这个过程其实很接近现在 AI 编程中强调的两个能力：一是提供足够准确的 context，让 AI 理解问题发生的环境；二是建立 Harness，给予 AI 约束规则、验证反馈闭环，让他的修改最小化，让每次修改都能被实际运行结果检验。AI 可以加快分析和修改速度，但人仍然需要读懂基本逻辑、观察实验现象、判断问题位置，并决定 AI 给出的修改是否真的适合当前任务。

（5）对传统图像处理流程的理解更完整了

通过这个过程，我也更理解了传统图像处理方法之间的联系。以前看高斯滤波、Sobel 算子、Otsu 二值化、形态学操作时，只觉得它们是一个个独立知识点；但真正做项目时才发现，它们其实是连在一条流程里的。

前一步处理不好，后面的车牌定位、字符分割和识别都会受影响。因此调试时不能只盯着最终输出结果，而要一层一层往前看，找到最早出问题的环节。比如这次绿牌识别能成功，并不是一次性改出来的，而是不断查看中间图像、发现问题、补充上下文、让 AI 协助分析，再逐步调整颜色判断、字符数量、分割规则和模板匹配范围，最后才得到正确结果。

（6）补充基础与文档整理也是项目的一部分

除了代码调试，我还负责了车牌识别部分的报告编写、部分实验内容整理，以及小组报告中相关内容的修改和统一。在整理组员提交的内容时，也需要检查实验结果、文字描述和最终结论是否一致，避免出现代码结果和报告表述不一致、不同模块表述风格不统一、实验分析不够清楚等问题。为了更好理解整个项目，我还在课外补充学习了吴恩达机器学习相关内容，对监督学习、损失函数、梯度下降、神经网络、偏差和方差、正则化、迁移学习等概念有了初步认识。这样再看组员的鸢尾花、猫狗分类和 MNIST 实验时，就能进一步理解为什么要划分训练集和测试集、为什么 CNN 适合图像、为什么预训练 VGG16 在小数据集上效果更好。